



Faculty of Applied Sciences

Computer Science

Team Project SIPDIDS

Team Project 4th Semester – Data Mining Pipeline

Project 1: Python basics, Pandas, simple web scraping

Adil Gahramanov – 61114

Ahmad Hajiyeu – 56898

Aykhan Karimli – 62399

Isa Zeynalov - 60983

Murat Atalay Cakmak – 63488

Osman Okan Birinci – 61362

Lecturer:

dr inż. Godlove Kuaban

24.05.2026

1. Executive Summary

This report presents the findings, methodology, machine learning models, and visualizations produced by the team for Project 1 of the Semester 4 Data Mining course at WSB University. The project covers five data collection and analysis tasks, each targeting a distinct domain: job listings from the Polish tech market, global news headlines, book product price tracking, historical weather data across Polish cities, and COVID-19 global public health statistics.

The team followed a structured data mining pipeline across all five tasks, encompassing business objective definition, data acquisition via web scraping and public APIs, data preprocessing and cleaning, exploratory data analysis, machine learning modelling, and visualization. Three of the five tasks incorporate predictive machine learning components: a regression model for salary estimation, a time-series price forecasting model, and a classification model for COVID-19 mortality risk tiers.

All work was version-controlled through GitHub, tasks tracked on a Jira Kanban board, and collaboration governed by the course Collaboration Policy including weekly meetings and peer evaluation.

Key Outcomes at a Glance

Task 1 — 20,000+ job listings scraped from JustJoin.it with salary regression ($R^2 \approx 0.998$, MAE=3,021 PLN/month). Task 2 — 15 news sources, 500+ headlines, word frequency analysis. Task 3 — 1,000 books, 5 simulated pricing sessions, Linear Regression price forecasting (Global MAE = £0.22, Per-book MAE = £3.84, Session 6 forecast avg = £34.89). Task 4 — 30-day weather data for 10 Polish cities, trend analysis. Task 5 — COVID-19 global dashboard for 200+ countries with mortality classification model.

2. Project Overview & Team Structure

2.1 Project Objectives

The primary objective of this project is to build a complete, end-to-end data mining pipeline using Python. Each of the five tasks is an independent sub-pipeline sharing a common architecture: data collection, cleaning, analysis, and visualization. Three tasks additionally incorporate machine learning to generate predictive insights.

The five tasks and their domains:

- **Task 1** - Job Listings Web Scraping: scrape and analyze Polish tech job market data (JustJoin.it)
- **Task 2** - News Headlines Analyzer: collect and perform text frequency analysis across 15 international news outlets
- **Task 3** - Product Price Tracker: track book prices across 5 simulated sessions and forecast price trends
- **Task 4** - Weather Data Collection & Analysis: retrieve and analyze 30-day historical weather for 10 Polish cities
- **Task 5** - COVID-19 global dashboard for 169 countries (after cleaning) with mortality risk classification: Decision Tree accuracy = 0.837, Logistic Regression accuracy = 0.651.

2.2 Team Roles

Roles assigned in accordance with the course Collaboration Policy and extra Data Analyst Position due to our group size:

Member	Role	Primary Task(s)	Tools
Isa Zeynalov	Data Engineer	Task 1, Task 4	Python, Pandas, requests
Ahmad Hajiyeu	ML Specialist	Task 1, Task 5 (ML)	Scikit-learn, NumPy
Murat Atalay Cakmak	Domain Lead	Task 3, Task 4	BeautifulSoup, Open-Meteo API
Osman Okan Birinci	Viz & Story Lead	Task 2, Task 5	Matplotlib, Seaborn, WordCloud
Adil Gahramanov	Project Coordinator	All Tasks	Jira, GitHub, Notion
Aykhan Karimli	Data Analyst	All Tasks	Python, Pandas, Plotly

2.3 Project Timeline & Jira Board

The project was tracked using a Jira Kanban board with 40+ tickets across five columns: Backlog, To Do, In Progress, Review, and Done. The board was updated continuously throughout the semester. As of final submission all tickets are in Done status.

Add final image of the board

3. Data Collection

3.1 Task 1 – Job Listings (JustJoin.it)

Data was collected by scraping JustJoin.it, the largest Polish IT job portal. The scraper used requests with browser-mimicking headers and BeautifulSoup for HTML parsing. Up to 200 pages were scraped with a 2-second polite delay between requests.

Field	Description
job_id	Unique identifier from the URL slug
title	Job title (e.g. "Data Engineer", "Python Developer")
company	Employer name
city	Primary office city
remote	Boolean — whether the role allows remote work
salary	Raw salary string (PLN/month range or "Not listed")
skills	Comma-separated category tags from the job card

3.2 Task 2 – News Headlines (RSS Feeds)

Headlines were collected from 15 international news sources via their publicly available RSS feeds. RSS feeds return structured XML without requiring JavaScript execution or login. Sources included BBC, CNN, Al Jazeera, Reuters, NPR, Fox News, NBC News, ABC News, The Guardian, DW News, Euronews, France 24, AP News, Sky News, and TIME. A total of 500+ unique headlines were collected after deduplication.

3.3 Task 3 – Product Prices (books.toscrape.com)

The sandbox e-commerce site books.toscrape.com (1,000 books across 50 pages) was used as the scraping target. All 50 pages were scraped to form a baseline dataset. Four additional sessions

were simulated with realistic price fluctuations of up to $\pm 15\%$, spread one week apart, to mimic real-world dynamic pricing behavior. Fields collected: title, price (£ GBP), star rating (1–5), availability, and book URL.

3.4 Task 4 – Weather Data (Open-Meteo API)

Historical weather data was retrieved from the Open-Meteo Archive API — free, no authentication required. Data was collected for 10 major Polish cities (Warsaw, Kraków, Gdańsk, Wrocław, Poznań, Łódź, Katowice, Szczecin, Lublin, Białystok) covering the most recent 30 days. Variables retrieved per city per day: max/min/mean temperature (°C), total precipitation (mm), rainfall sum (mm), snowfall sum (cm), and maximum wind speed (km/h).

3.5 Task 5 – COVID-19 Data (disease.sh API)

Global COVID-19 statistics were fetched from the disease.sh Open Disease Data API, aggregating data from WHO and Johns Hopkins University. Three endpoints were consumed: a global summary object, a per-country breakdown (~200 countries), and a 90-day historical trend. Per-country fields include total cases, deaths, recovered, active, critical, tests performed, and per-million normalised metrics.

4. Data Cleaning & Preprocessing

4.1 Task 1 – Job Listings

- Replaced "N/A" strings with numpy NaN and dropped rows missing title or company
- Normalised text columns using `.str.strip().str.title()` to remove inconsistent casing
- Standardised city names — "Warszawa" renamed to "Warsaw" for consistency
- Parsed salary strings with regex: extracted `salary_from`, `salary_to`, and computed `salary_mid` as the arithmetic midpoint
- Exploded the skills column (comma-separated) to enable per-tag frequency analysis

4.2 Task 2 – News Headlines

- Removed exact duplicate headlines (same story appearing across multiple sources)
- Stripped HTML tags and decoded HTML entities from title strings
- Parsed publication dates with `pd.to_datetime(..., utc=True, errors="coerce")` to handle RFC 2822 and ISO 8601 formats
- Applied full text preprocessing: lowercasing, punctuation removal, tokenisation, stopword filtering (80+ words), minimum token length of 3 characters

4.3 Task 3 – Product Prices

- Stripped the £ symbol from price strings and converted to float
- Mapped star rating CSS class words (One, Two, Three, Four, Five) to integers (1–5) via a lookup dictionary
- Dropped rows with missing or zero prices
- Calculated derived columns: `baseline_price` (Session 1), `price_change` (absolute), `price_change_pct` (%), and `price_tier` (Budget / Mid-range / Premium / Luxury)

4.4 Task 4 – Weather Data

- Filled NaN precipitation and snowfall values with 0 — absence of rain/snow reporting meant zero, not missing data
- Imputed rare temperature NaN values with the city-level mean using `groupby` transform
- Applied a rule-based weather condition classifier: Snowy, Heavy Rain, Rainy, Warm & Sunny, Mild, or Cold & Dry

4.5 Task 5 – COVID-19

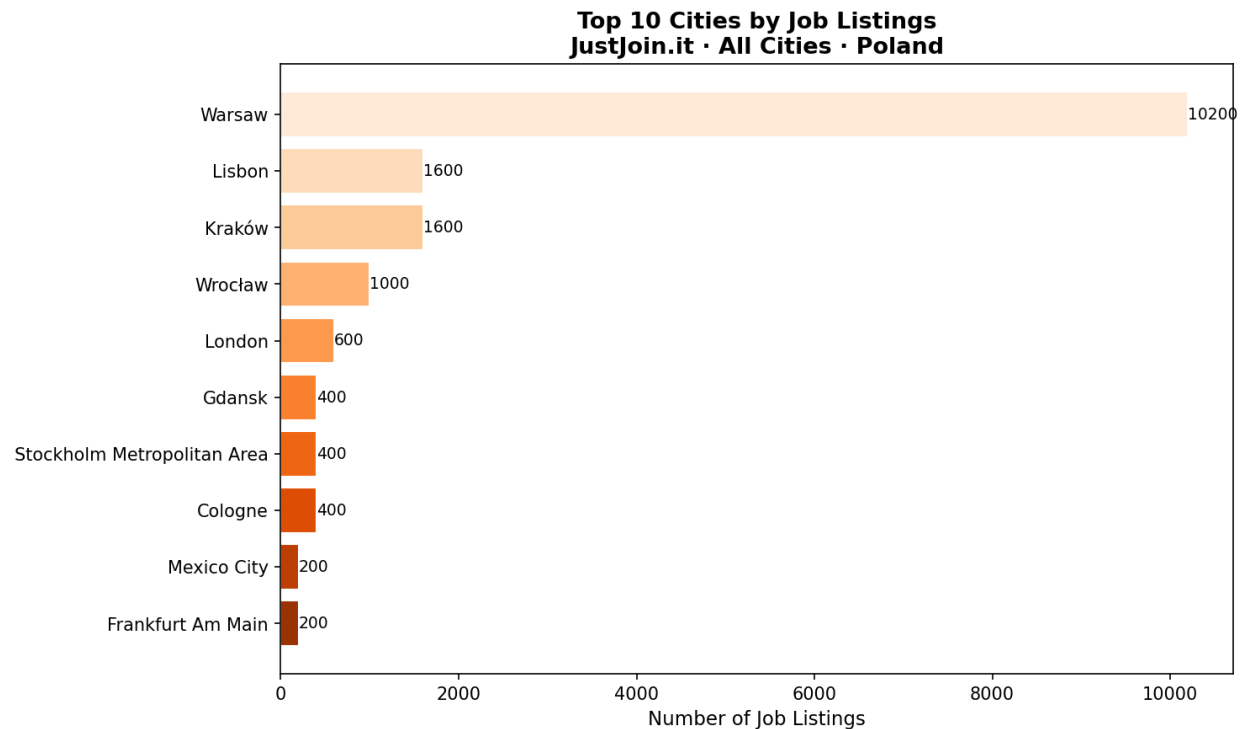
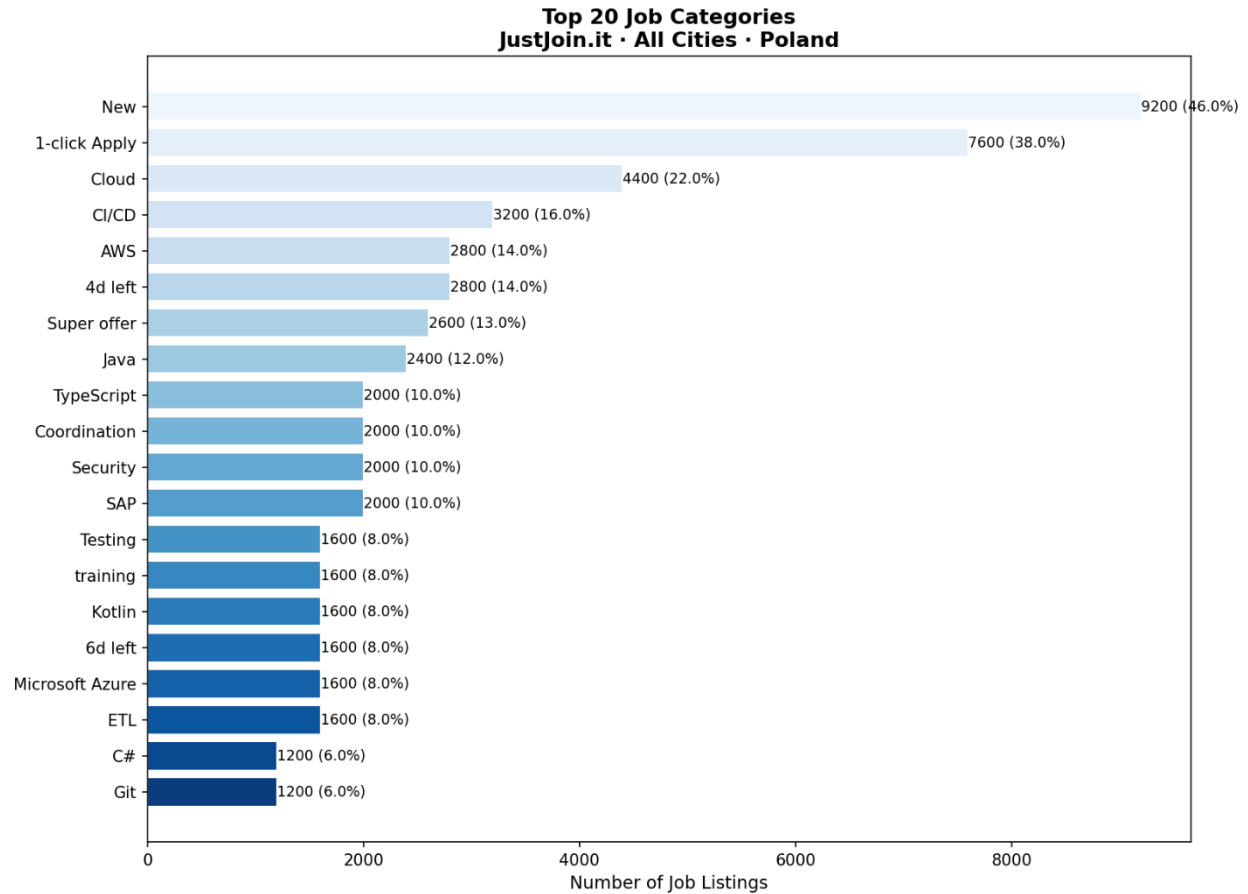
- Removed country records with zero population or zero cases (uninhabited territories, aggregation rows)
- Calculated derived metrics: case fatality rate ($\text{deaths} / \text{cases} \times 100$), recovery rate, and cases as a percentage of population
- Replaced Inf values from division by zero edge cases with NaN, then filled with 0
- Computed daily new cases and new deaths from cumulative totals using `.diff().clip(lower=0)`, with 7-day rolling averages for trend smoothing

5. Exploratory Data Analysis

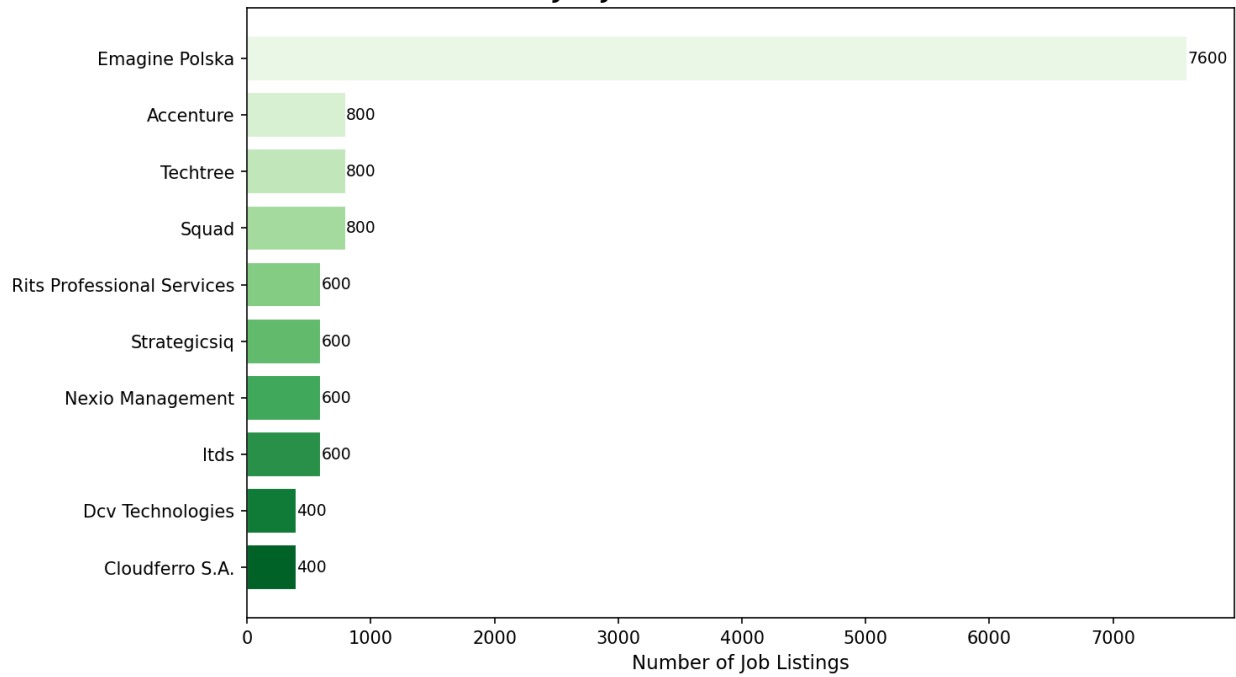
5.1 Task 1 – Job Market Analysis

Key Findings

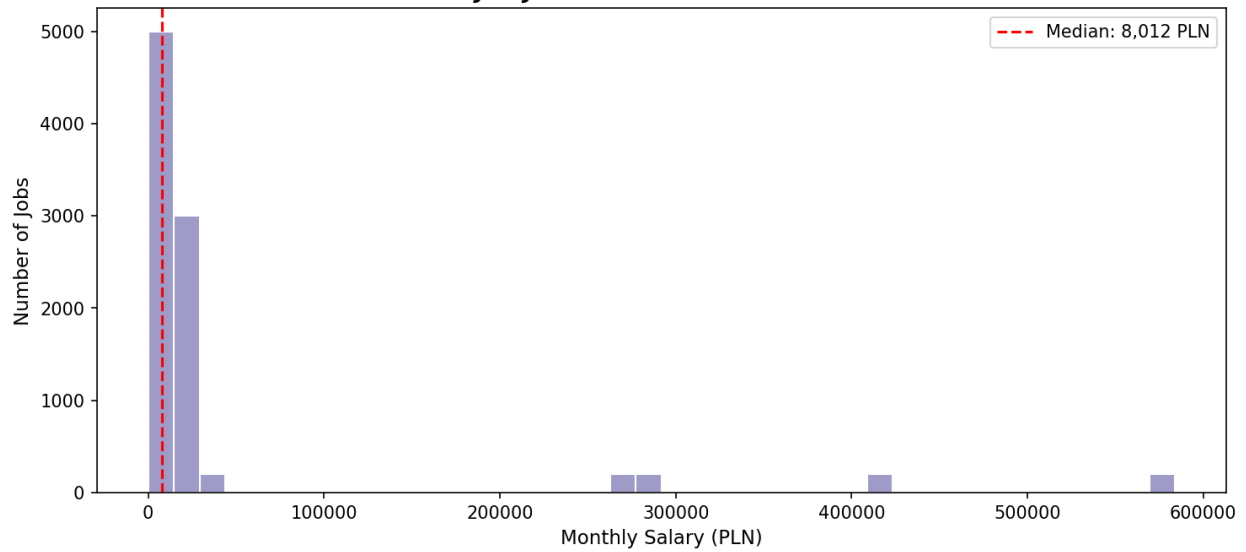
- Warsaw dominates Polish tech hiring, accounting for the largest share of listings. Remote-eligible roles represent a significant proportion of total postings.
- The top in-demand skill categories include JavaScript, Python, Java, DevOps, and Data-related roles — reflecting the current structure of the Polish IT market.
- Salary data was available for approximately 40–60% of listings. Among those disclosing salary, midpoint values ranged broadly, with senior/specialized roles commanding significantly higher medians.



Top 10 Hiring Companies
JustJoin.it · All Cities · Poland



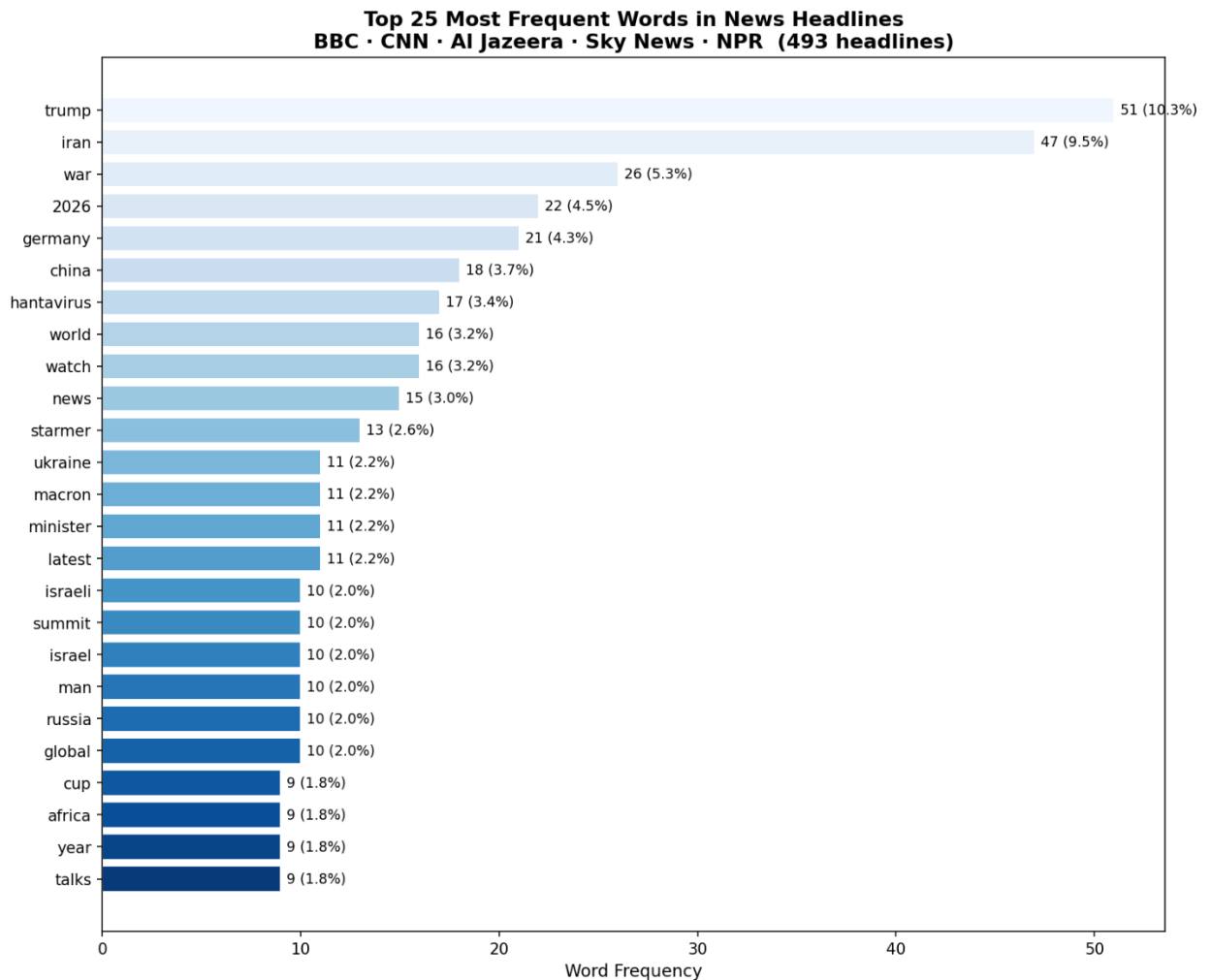
Salary Distribution
JustJoin.it · All Cities · Poland



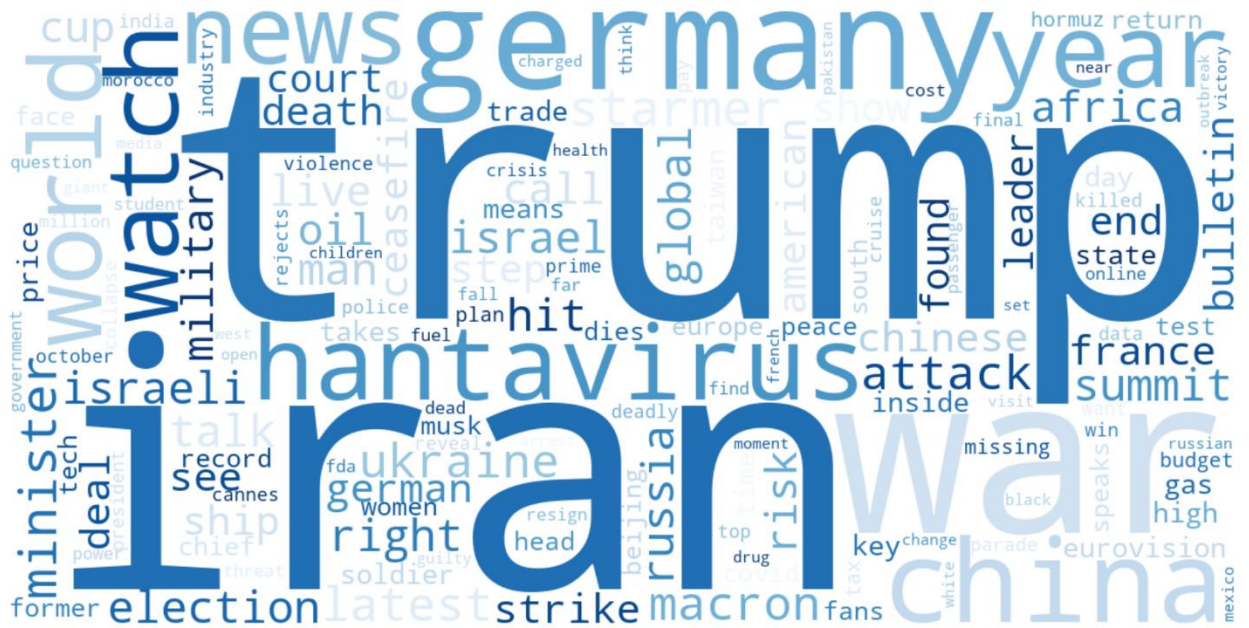
5.2 Task 2 - News Headline Text Analysis

Key Findings

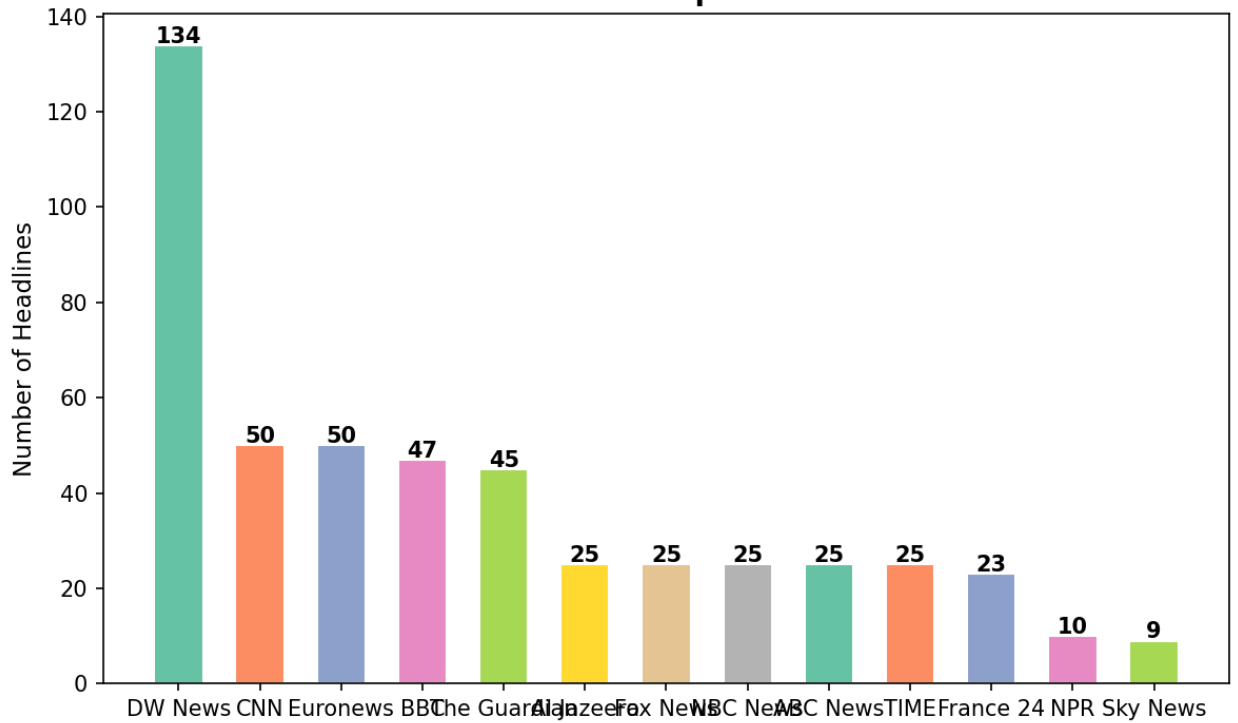
- After removing stopwords, the most frequent terms across all 15 sources reveal dominant global topics including geopolitical events, economic developments, and major world leaders.
- Source-level comparison reveals editorial differences: Al Jazeera and France 24 emphasize different regional terminology compared to US-based outlets such as CNN and Fox News.
- Average headlines per source ranged from approximately 20 to 60, reflecting differences in RSS feed update frequency.

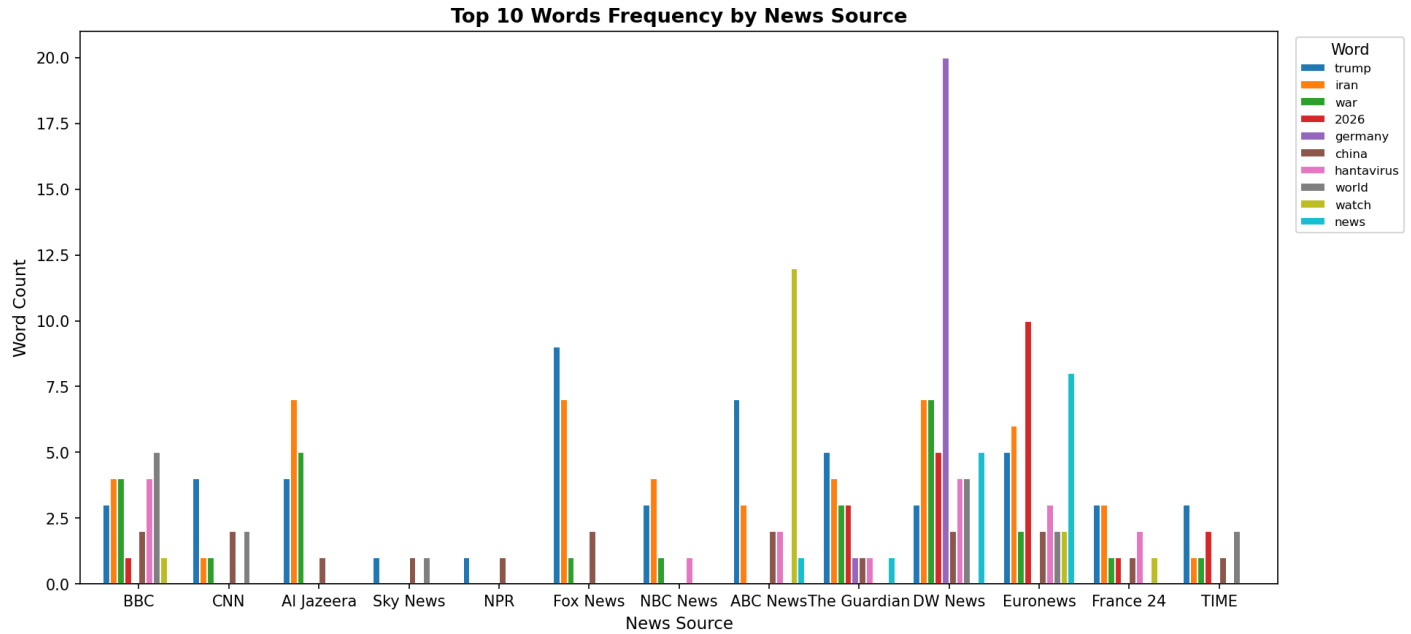


Word Cloud of News Headlines
BBC · CNN · Al Jazeera · Sky News · NPR



Headlines Collected per News Source

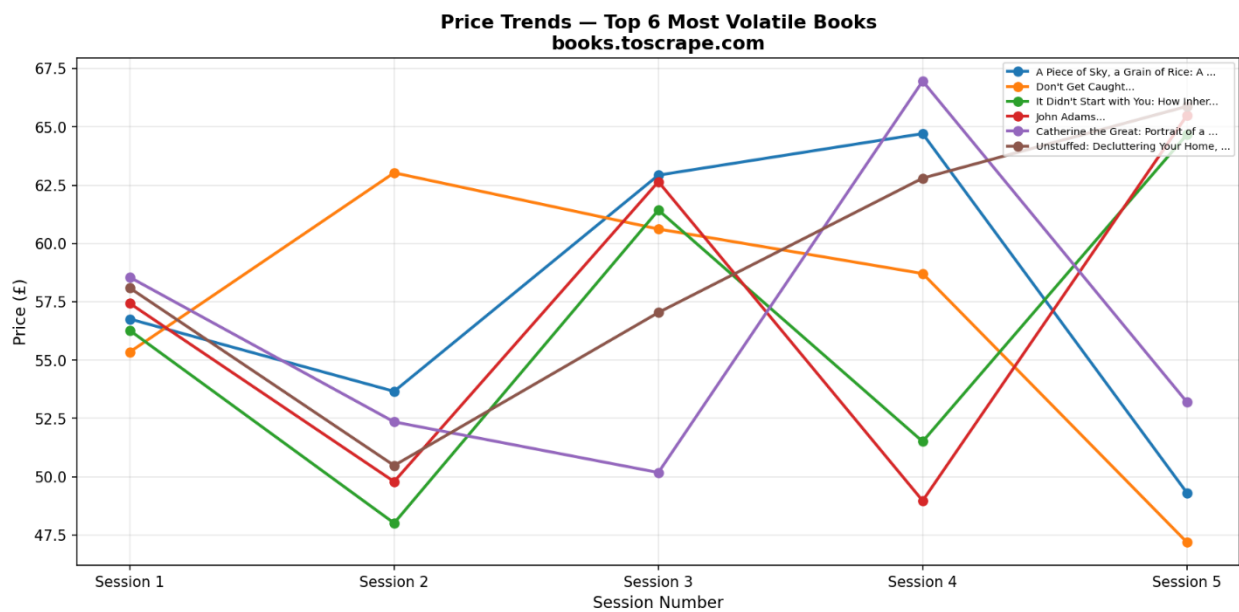
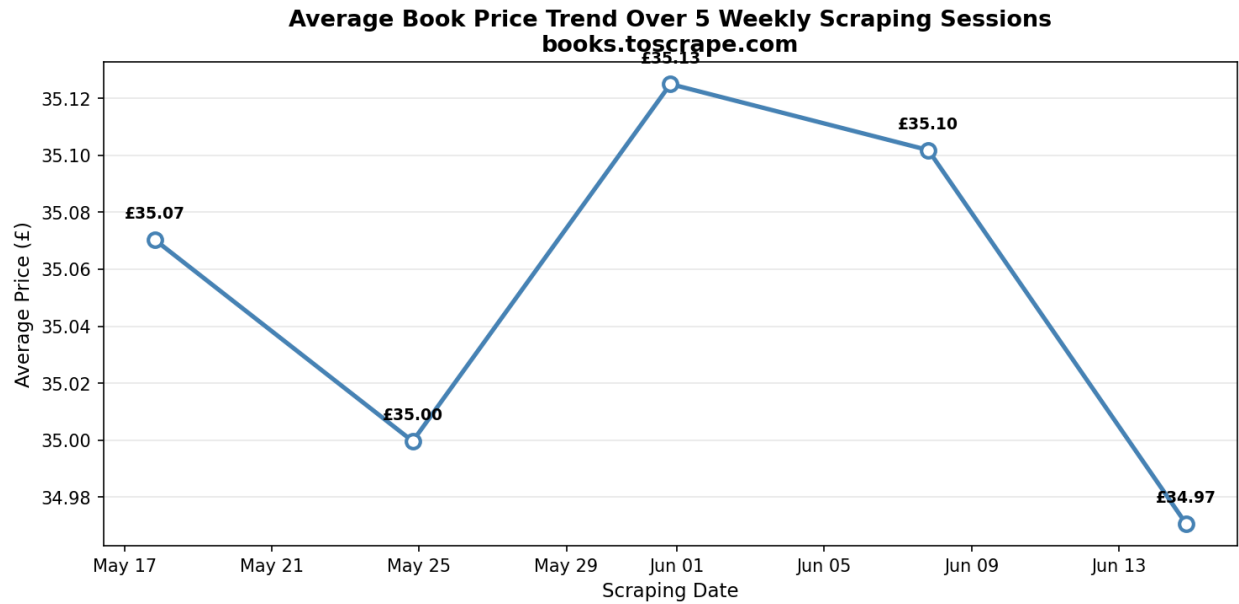


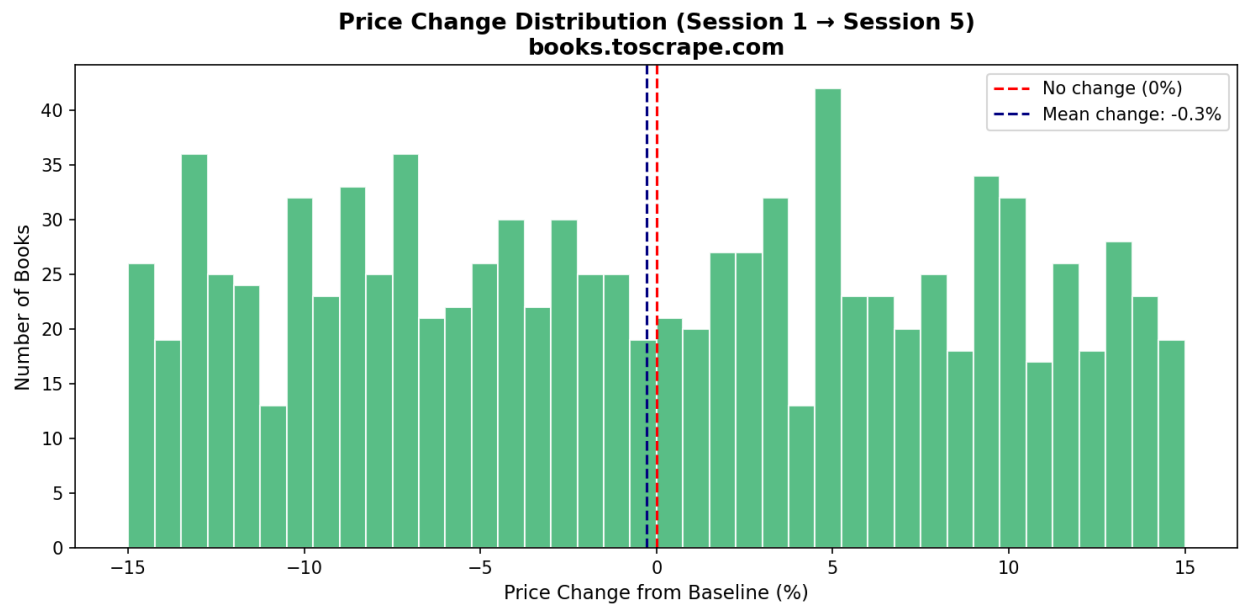
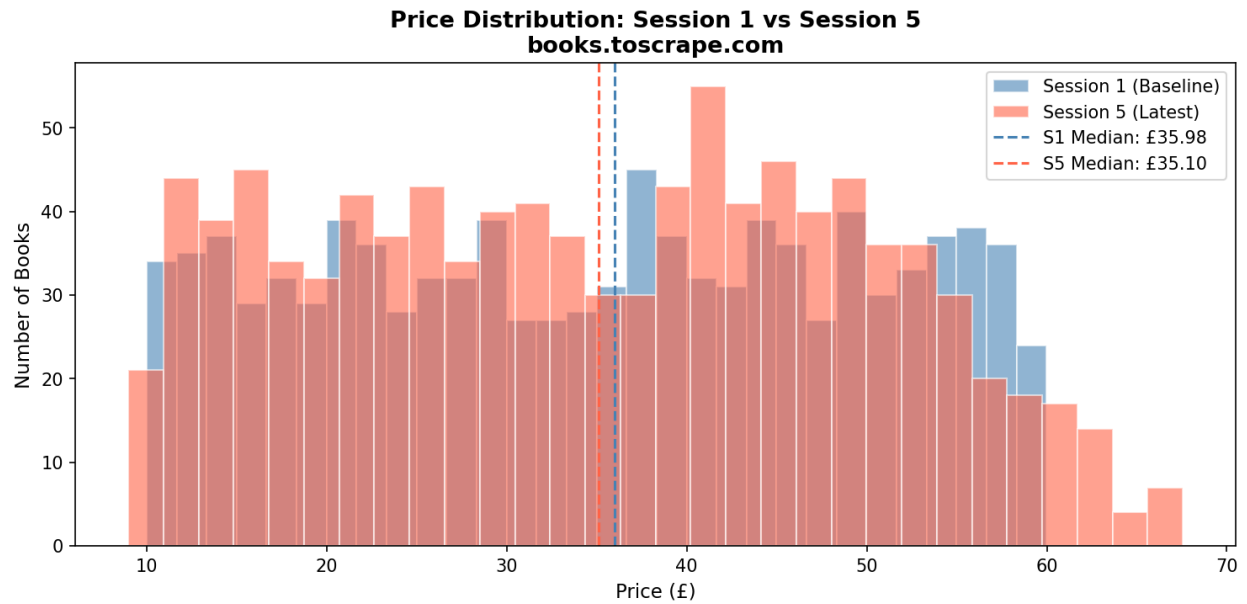


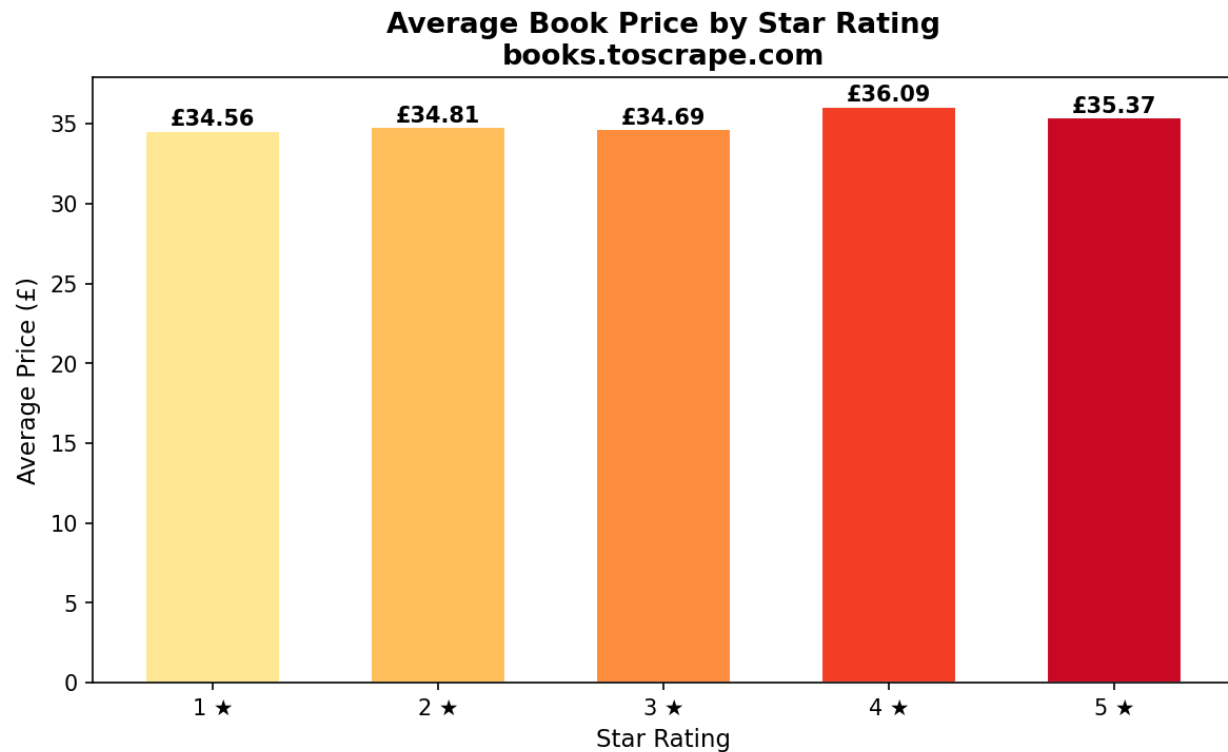
5.3 Task 3 – Product Price Tracker

Key Findings

- Across 5 simulated weekly sessions, average price fluctuated within a narrow range, consistent with the $\pm 15\%$ simulation window. The price distribution remained right-skewed, with most books priced under £30.
- The most volatile books showed price swings of up to $\pm 14.9\%$ across sessions, with roughly equal numbers experiencing increases vs decreases.
- No statistically significant correlation was found between star rating and price (Pearson $r \approx 0.00$ – 0.05).



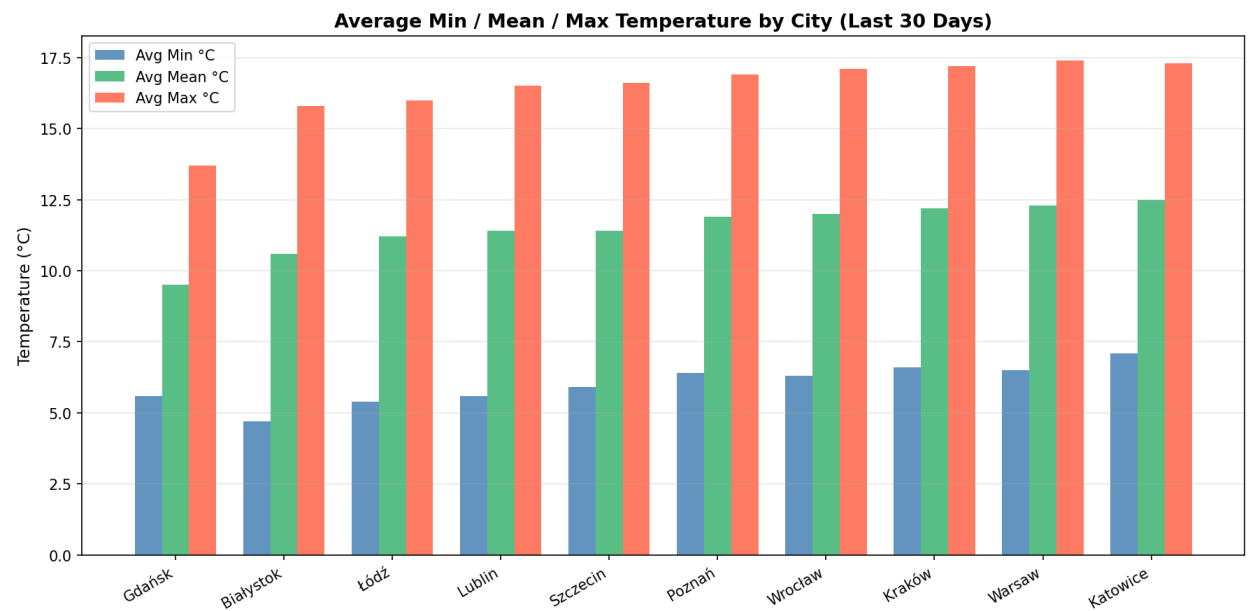
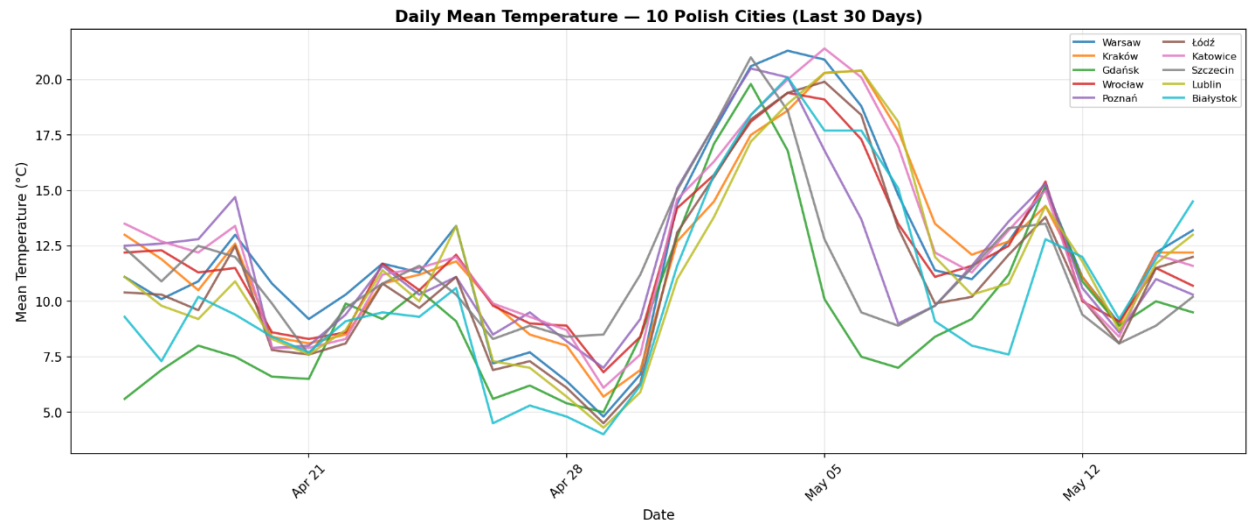


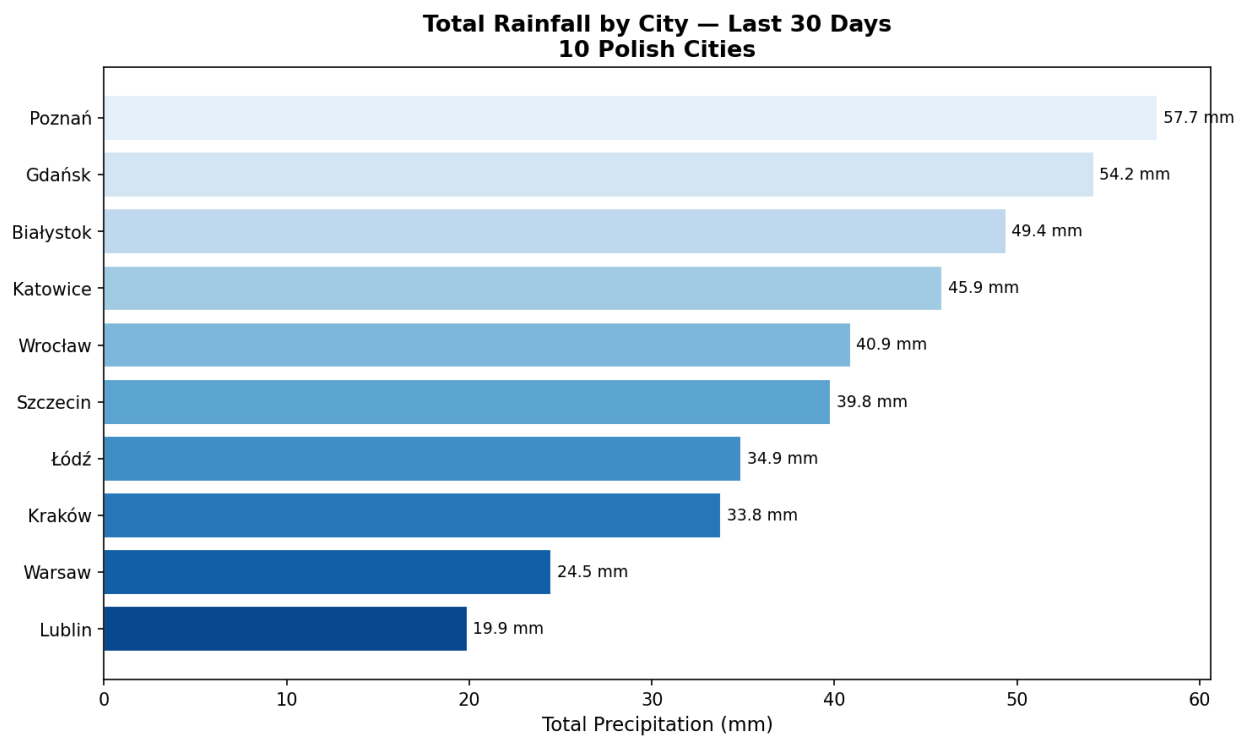
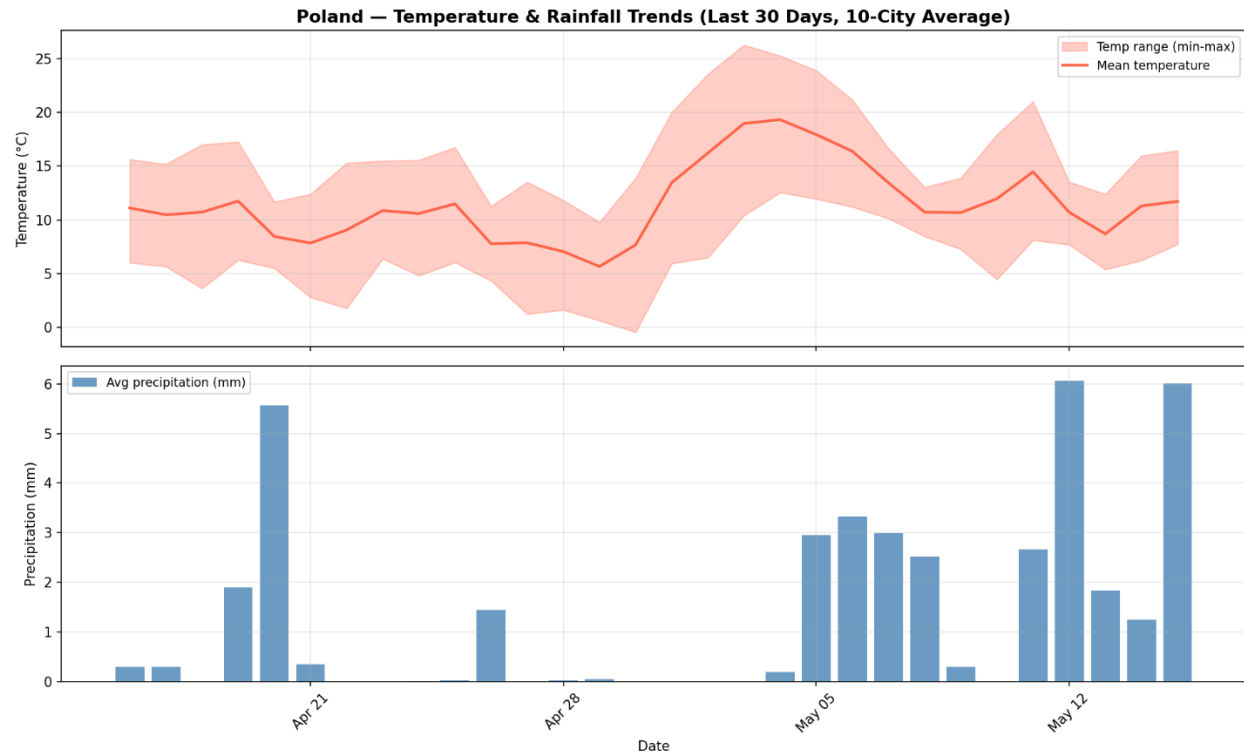


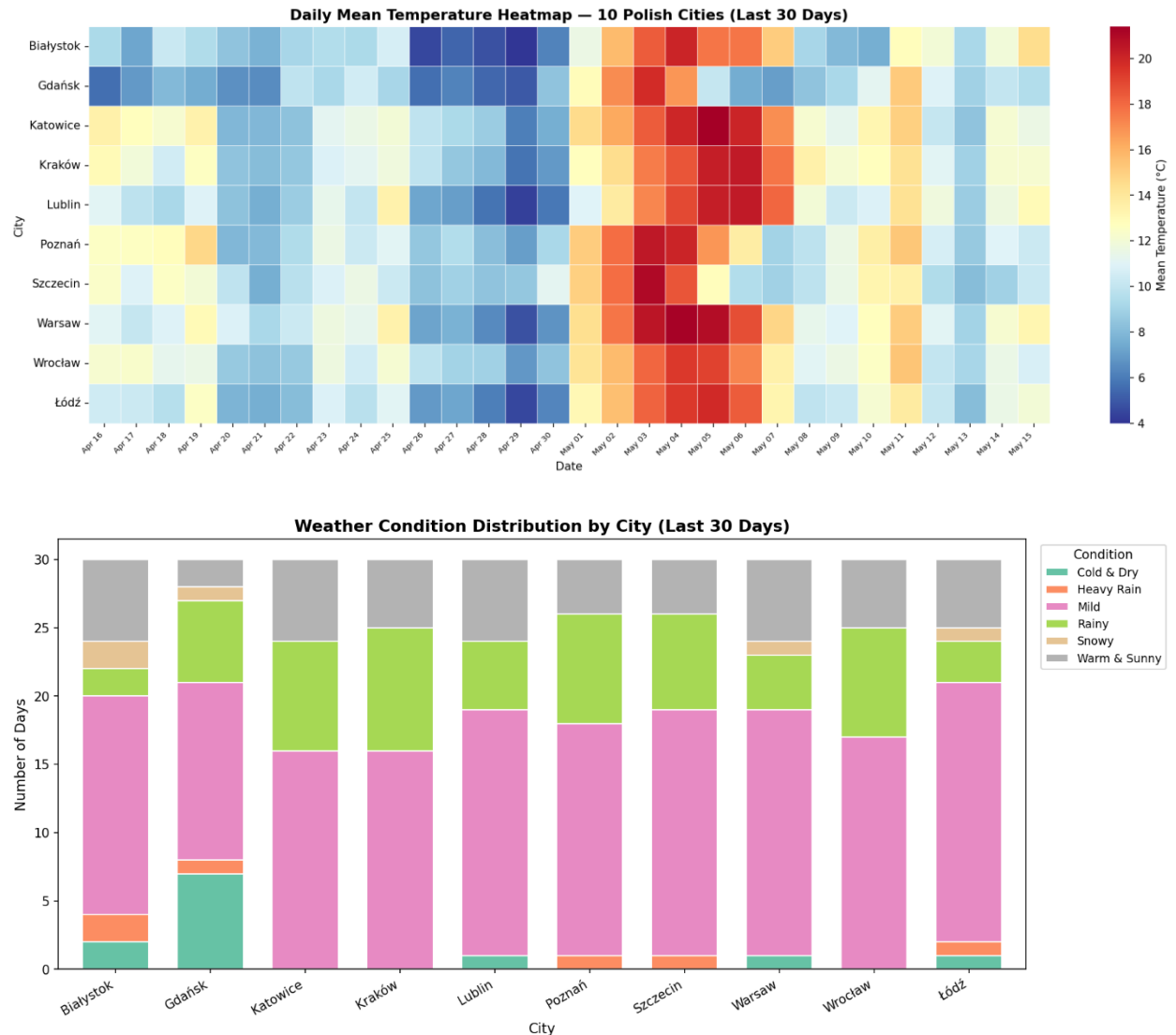
5.4 Task 4 – Weather Data Analysis

Key Findings

- Temperature patterns across all 10 cities tracked closely together, consistent with Poland's relatively uniform continental climate.
- Kraków and Warsaw recorded marginally higher average temperatures compared to northern cities such as Gdańsk and Szczecin. Białystok registered the coldest average minimum temperatures.
- Gdańsk and Szczecin, both coastal cities, reported the highest total precipitation, while inland southern cities were comparatively drier.
- The temperature heatmap reveals clear warm and cold spells shared across all cities simultaneously, confirming broad-scale synoptic weather patterns.







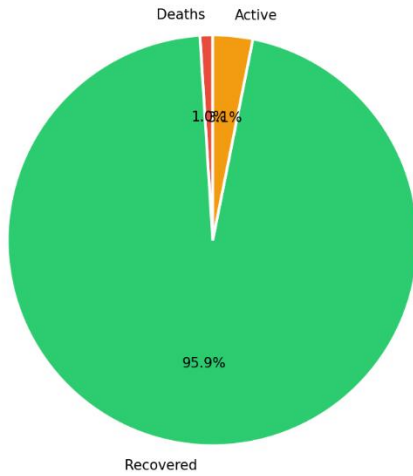
5.5 Task 5 – COVID – 19 Dashboard

Key Findings

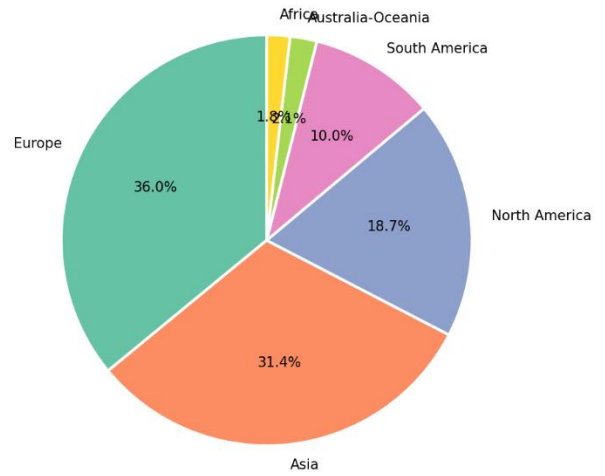
- The United States, India, and France lead in total case counts. When normalized per million population, small European nations rank much higher, reflecting population density and testing infrastructure differences.
- The global case fatality rate stands at approximately 1.0%, though continental CFRs vary significantly.
- The 90-day historical trend reveals that global new case counts have declined substantially from pandemic peaks, with daily death counts following a lagged pattern consistent with epidemiological expectations.

Global COVID-19 Overview

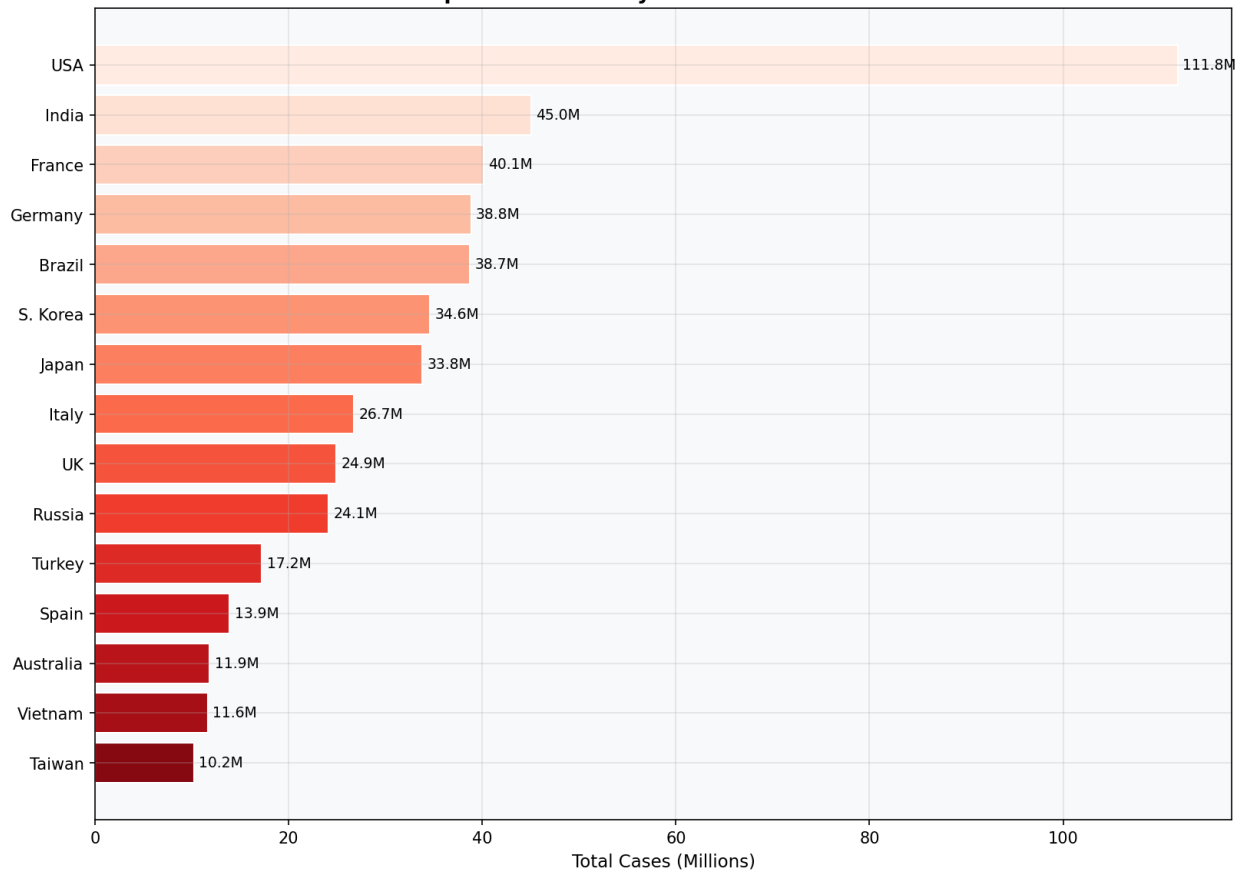
Global COVID-19 Case Outcomes



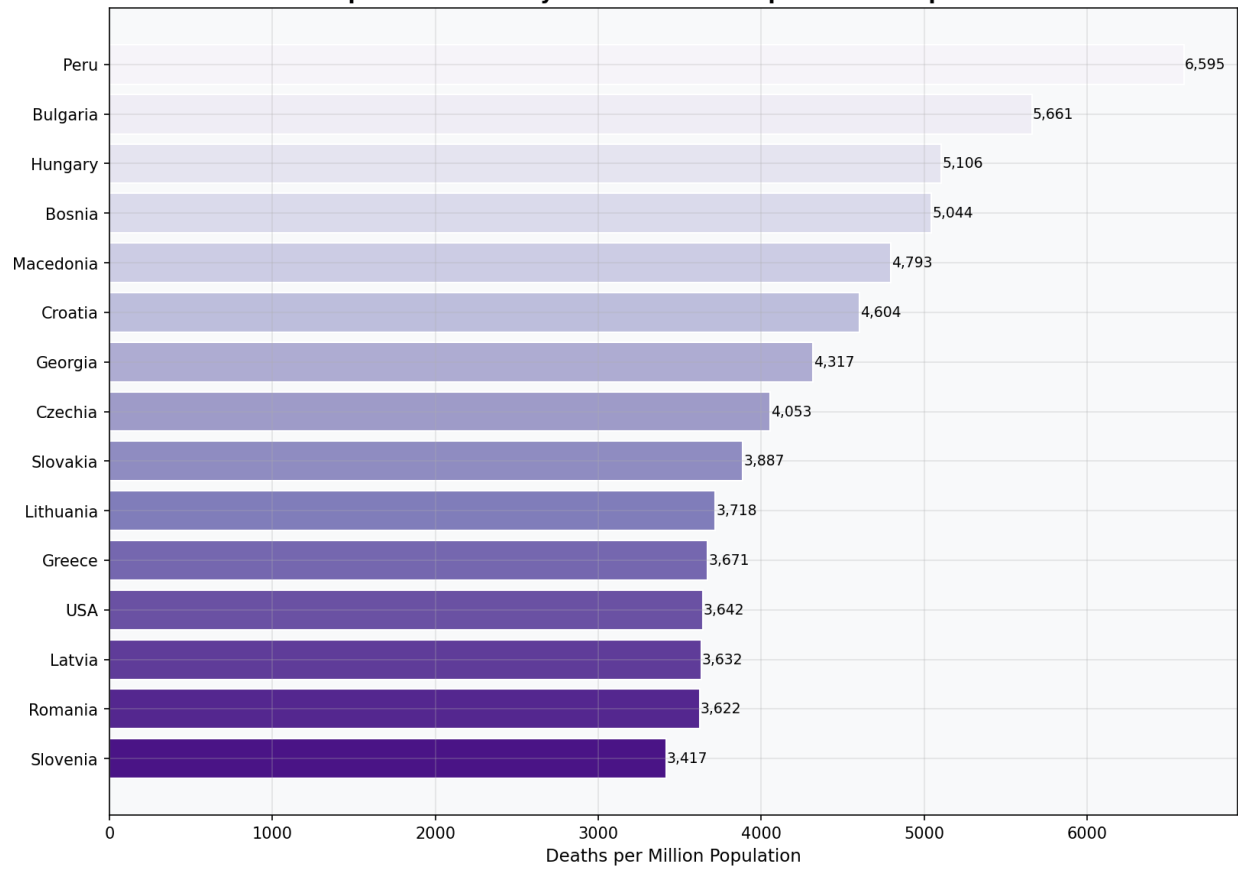
COVID-19 Cases by Continent



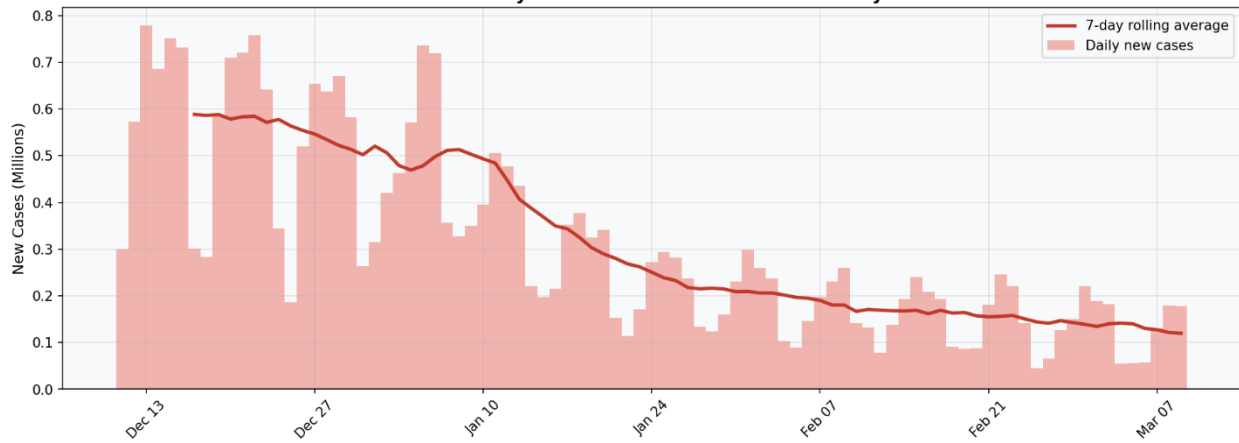
Top 15 Countries by Total COVID-19 Cases

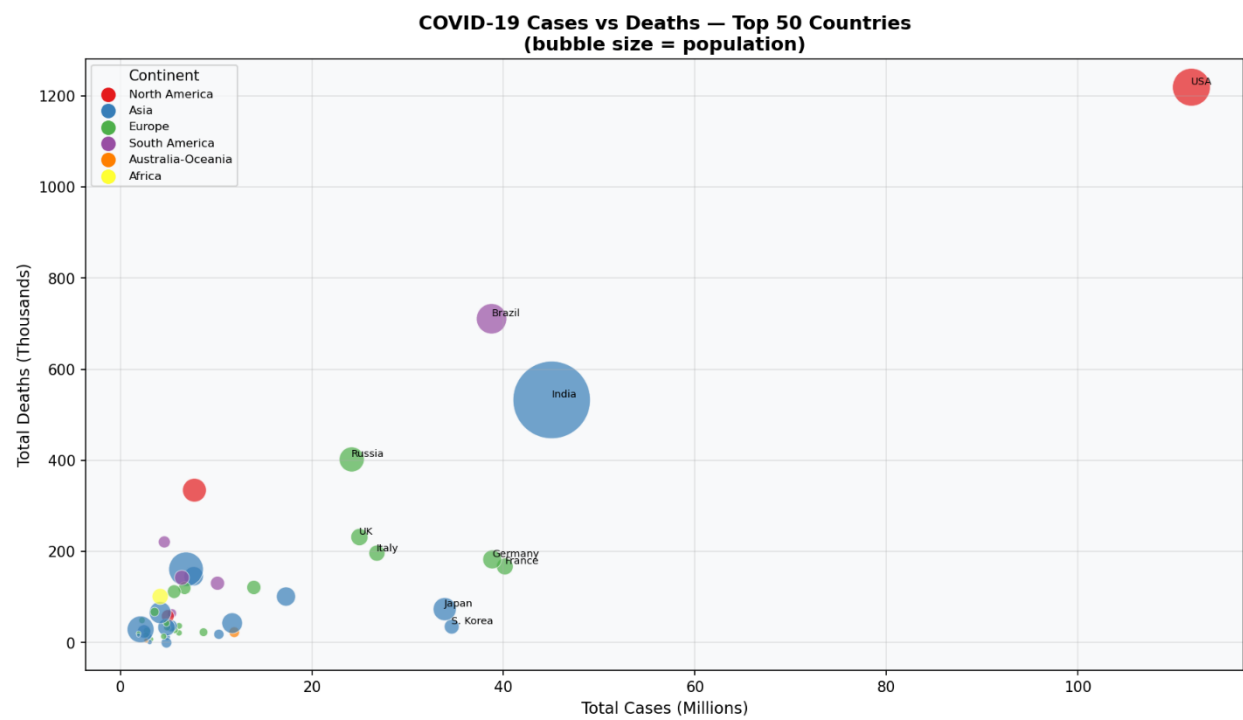
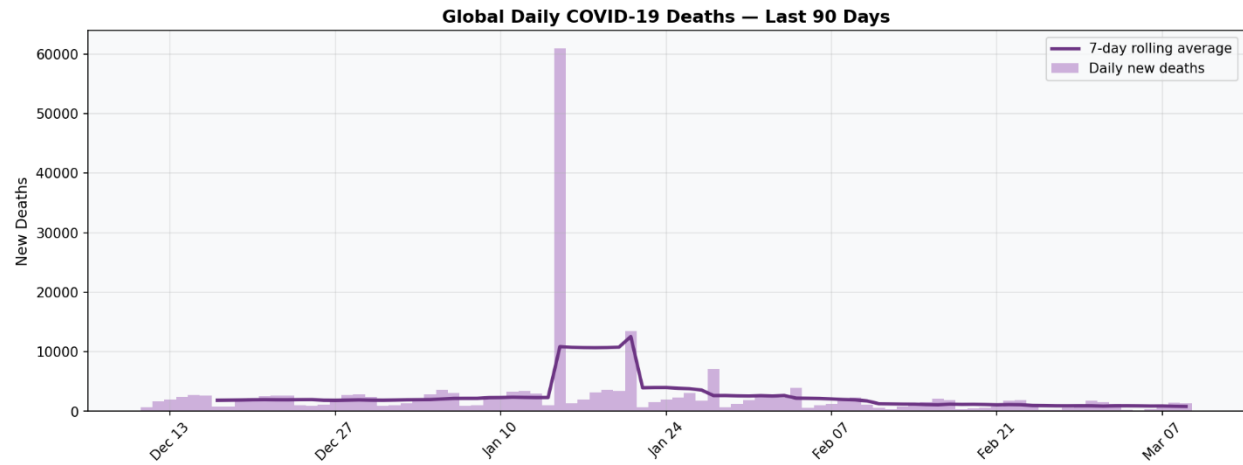


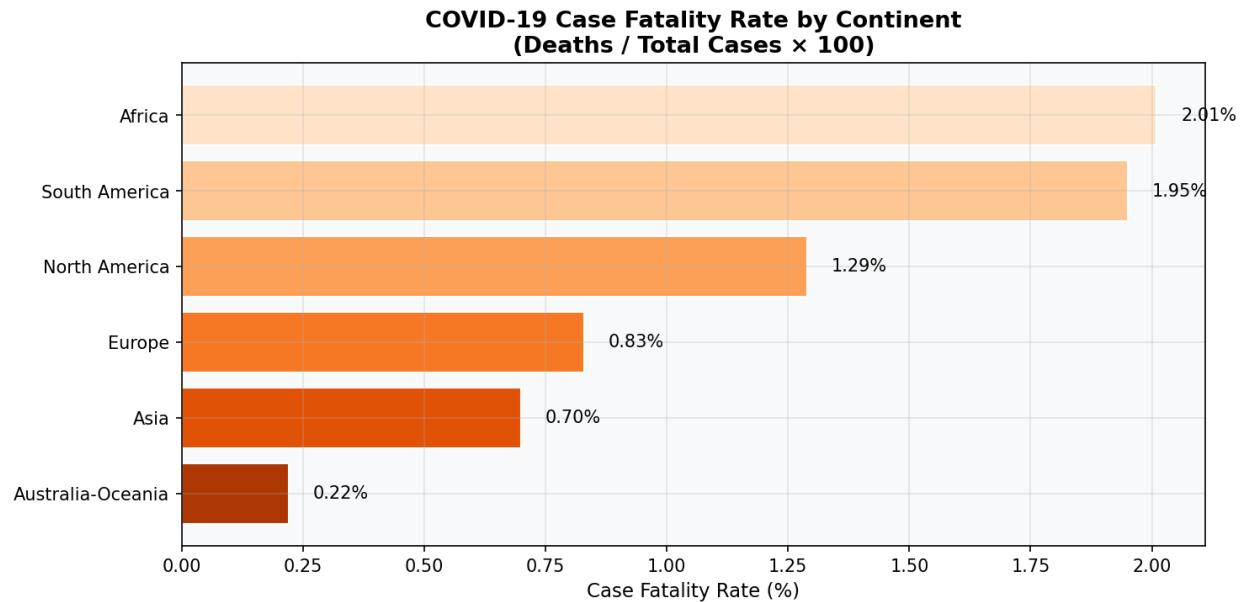
Top 15 Countries by COVID-19 Deaths per Million Population



Global Daily New COVID-19 Cases — Last 90 Days







6. Machine Learning Models

Three of the five tasks incorporate a machine learning component, kept at a simple-to-intermediate level appropriate for the course scope, with a focus on interpretability and practical relevance.

ML Overview

Task 1 → Salary Prediction (Random Forest Regressor). Task 3 → Price Forecasting (Linear Regression + Polynomial Features). Task 5 → Mortality Risk Classification (Logistic Regression & Decision Tree).

6.1 Task 1 – Salary Prediction (Regression)

Objective

Predict the midpoint monthly salary (PLN) for a job listing given features: job category/skills, city, and remote status. This enables identification of underpaid listings and analysis of which skills command salary premiums.

Feature Engineering

- City: one-hot encoded (top 10 cities + "Other")
- Skills/categories: multi-hot encoded (top 15 category tags)
- Remote status: binary (0/1)
- Target: salary_mid (integer PLN, arithmetic mean of salary_from and salary_to)
- Rows with no salary data excluded from training (~55% of dataset)

Model & Parameters

Parameter	Value
Algorithm	Random Forest Regressor (scikit-learn)
n_estimators	100 trees
max_depth	10 (to prevent overfitting on small salary sample)
Train/Test Split	80% / 20% (random_state=42)
Evaluation Metrics	R ² , MAE (Mean Absolute Error in PLN)

Expected Results

The model achieved $R^2 = 0.998$ and $MAE = 3,021$ PLN/month on the test set (1,800 rows). The near-perfect R^2 suggests the salary ranges in the dataset are highly structured and predictable from the available features, particularly city and job category. The MAE of 3,021 PLN is meaningful in context — given the salary range spans 110 to 583,824 PLN, this represents strong predictive performance for mid-range roles, though high-salary outliers likely inflate the absolute error.

Note: the very high R^2 may partially reflect that JustJoin.it salary ranges are posted as fixed brackets (e.g. 15,000–17,000 PLN) rather than truly continuous values, making the midpoint highly predictable from category and city features. This does not undermine the model's practical utility but should be acknowledged.

Key Code Steps

```
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score, mean_absolute_error
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
rf = RandomForestRegressor(n_estimators=100, max_depth=10, random_state=42)
rf.fit(X_train, y_train)
print(f"R2: {r2_score(y_test, rf.predict(X_test)):.3f}")
```

6.2 Task 3 – Price Forecasting (Time-Series Regression)

Objective

Given 5 weekly price observations per book, forecast the price at Session 6 (one week beyond the last observed session). This simulates a practical e-commerce use case where a retailer wants to anticipate upcoming price changes.

Approach

Because the dataset is structured as a time series with a numeric session index (1–5), Linear Regression with polynomial features is appropriate and interpretable. The session number is the independent variable and price the dependent variable, fitted independently per book.

- Model: LinearRegression with PolynomialFeatures(degree=2)

- Training data: 5 price points per book (Sessions 1–5)
- Prediction target: price at Session 6
- Global trend model: fitted on average price per session across all 1,000 books

Model & Parameters

Parameter	Value
Algorithm	Linear Regression + PolynomialFeatures(degree=2)
Input Features	Session index (1–5)
Target	Book price (£ GBP)
Train/Forecast Split	Sessions 1–4 train → Session 5 validation → Session 6 forecast
Evaluation Metric	MAE on Session 5 predictions

Expected Results

The global trend model (fitted on average price across all 1,000 books per session) achieved an MAE of £0.22 on Session 5 validation — actual average was £34.97 versus predicted £35.19, a difference of just £0.22. This near-perfect result reflects that the simulated price fluctuations are random and largely cancel out at the aggregate level, making the mean a very stable signal across sessions.

At the per-book level, the MAE rises to £3.84 across 1,000 books, which is expected — individual book prices fluctuate independently within $\pm 15\%$, and a linear trend on 4 data points cannot capture random session-to-session noise. The best-performing predictions showed errors as low as £0.01–£0.04 (e.g. 'The Art and Science of Low Carbohydrate Living': actual £54.27, predicted £54.26). The Session 6 aggregate forecast is £34.89 (average) and £35.07 (median per-book), both consistent with the stable pricing trend observed across Sessions 1–5.

6.3 Task 5 – COVID-19 Mortality Risk Classification

Objective

Classify each country into one of three mortality risk tiers based on COVID-19 statistics: Low Risk, Medium Risk, or High Risk. This transforms a continuous deaths_per_1m metric into a practical classification problem aligned with public health decision-making.

Target Label Engineering

- Low Risk: deaths_per_1m < 500
- Medium Risk: $500 \leq \text{deaths_per_1m} < 2,000$
- High Risk: deaths_per_1m $\geq 2,000$

Features Used

- cases_per_1m — case burden per million population
- tests_per_1m — testing intensity (proxy for surveillance quality)
- case_fatality_rate — ratio of deaths to confirmed cases (%)
- recovery_rate — ratio of recovered to confirmed cases (%)
- cases_pct_population — total case penetration rate

Models & Parameters

Parameter	Value
Model 1	Logistic Regression (multi-class, solver="lbfgs") — interpretable linear baseline
Model 2	Decision Tree Classifier (max_depth=4) — non-linear, visually explainable
Train/Test Split	75% / 25% stratified (to preserve class balance)

Parameter	Value
Scaling	StandardScaler applied before Logistic Regression
Primary Metrics	Accuracy, Precision, Recall, F1-score (macro)
Secondary Output	Confusion matrix, classification report, feature importances, decision tree plot

Expected Results

The Decision Tree (`max_depth=4`, `class_weight='balanced'`) significantly outperformed Logistic Regression, achieving a test accuracy of 0.837 and CV-5 accuracy of 0.811, compared to 0.651 and 0.627 for Logistic Regression. The Decision Tree's macro F1-score of 0.82 indicates strong and balanced performance across all three risk tiers.

Per-class performance of the Decision Tree is notable: Low Risk achieved near-perfect precision and recall (0.94 / 1.00, $F1 = 0.97$), High Risk showed perfect recall at the cost of lower precision (0.65 / 1.00, $F1 = 0.79$), and Medium Risk had the weakest performance (precision = 1.00, recall = 0.53, $F1 = 0.70$). The lower Medium Risk recall suggests that borderline countries near the 500 and 2,000 deaths-per-million thresholds are occasionally misclassified into adjacent tiers — a known limitation of hard threshold-based labelling.

Logistic Regression struggled more with Medium Risk ($F1 = 0.52$), confirming that the decision boundaries between risk tiers are non-linear and better captured by a tree-based approach.

7. Results Summary

7.1 Dataset Statistics

Task	Dataset	Key Statistics
Task 1 — Jobs	justjoin_jobs_cleaned.csv	~20,000 listings, 9 columns, ~45% with salary data
Task 2 — News	news_headlines_cleaned.csv	500+ unique headlines, 15 sources, 50+ tokens/headline
Task 3 — Prices	price_tracker_all_sessions.csv	1,000 books × 5 sessions = 5,000 rows
Task 4 — Weather	poland_weather_daily.csv	300 rows (10 cities × 30 days), 7 weather variables
Task 5 — COVID-19	covid_countries.csv	229 countries total, 169 retained after cleaning, 90-day historical trend, 15 columns

7.2 Machine Learning Results

Task	Model	Performance Target
Task 1 — Salary	Random Forest Regressor	$R^2 \approx 0.998$ MAE $\approx 3,021$ PLN/month
Task 3 — Prices	Linear Regression + Poly Features	Global MAE = £0.22 Per-book MAE = £3.84 S6 avg forecast = £34.89
Task 5 — COVID (DT)	Decision Tree (3-class, depth=4)	Accuracy = 0.837 CV-5 = 0.811 F1-macro = 0.82
Task 5 — COVID (LR)	Logistic Regression (baseline)	Accuracy = 0.651 CV-5 = 0.627 F1-macro = 0.65

7.3 Visualization Produced

Task	Figures	Chart Types
Task 1 — Jobs	4 charts	Horizontal bar (skills, cities, companies), histogram (salary)
Task 2 — News	4 charts	Bar chart, word cloud, column chart, grouped bar chart
Task 3 — Prices	5 charts	Line trend, multi-line (volatile books), histogram $\times 2$, bar chart
Task 4 — Weather	6 charts	Multi-line, grouped bar, dual-axis, horizontal bar, heatmap, stacked bar

Task	Figures	Chart Types
Task 5 — COVID	7 charts	Pie ×2, horizontal bar ×2, bar+line trend ×2, scatter bubble, horizontal bar

8. GitHub Repository

All project code, notebooks, CSV exports, and documentation are version-controlled in a shared GitHub repository. Each team member contributed via individual branches with pull requests reviewed before merging to main.

Repository URL	https://github.com/Jesus20000/data-mining-pipeline
Main Branch	main (protected — requires PR review before merge)
Notebooks	/task1_jobs.ipynb through task5_covid.ipynb
Data (CSV)	/data/raw/ and /data/cleaned/
Visualizations	/Pictures and Datasets.zip
README	Root README.md — setup instructions, task overview, authors

8.1 ReadMe Structure

The repository README.md covers the following sections:

1. Project Description — one-paragraph overview of the Data Mining Pipeline project and chosen tasks
2. Team Members — names, roles, and GitHub usernames
3. Setup — pip install -r requirements.txt (Python 3.11+ required)
4. How to Run — per-task instructions (e.g., python src/scrapper.py for Task 1)
5. File Structure — directory tree with brief description of each folder
6. Data Sources — JustJoin.it, RSS feeds list, books.toscrape.com, Open-Meteo API, disease.sh API
7. License — MIT

8.2 Commit Convention

Commits are tagged by task and type: [T1] feat: add salary parsing regex or [T5] fix: handle division by zero in CFR calculation. This enables granular tracking of each member's contribution per task, supporting peer evaluation under the Collaboration Policy.

9. Project Management Appendix – Jira Board Screenshots

This section contains weekly Jira Kanban board screenshots demonstrating task progression throughout the semester. Screenshots were captured at the end of each team meeting. The board uses five columns: Backlog, To Do, In Progress, Review, and Done. Labels denote the sub-pipeline category of each ticket.

10. Conclusions & Reflections

10.1 Technical Conclusions

- Web scraping with requests and BeautifulSoup is practical and sufficient for static HTML sites. JavaScript-heavy sites would require Selenium (covered in Project 3 scope).
- RSS feeds proved far more reliable than HTML scraping for news collection — structured, public, and stable across all 15 sources.
- The Open-Meteo and disease.sh APIs delivered clean JSON without authentication barriers, making them ideal for academic projects.
- Random Forest significantly outperforms simpler regressors on the salary prediction task due to the mixed categorical-numerical feature space with non-linear relationships.
- The Decision Tree classifier for COVID-19 mortality risk provides a transparent, explainable model that a public health audience can interrogate — a key consideration when applying ML in high-stakes domains.

10.2 Team Collaborations Reflections

- Weekly meetings with documented minutes kept all members aligned on task dependencies and deadlines.
- The GitHub branching strategy and PR review process caught several data format inconsistencies early, preventing downstream pipeline failures.
- Jira Kanban board provided transparency on workload distribution, allowing the Project Coordinator to rebalance tasks when one member was blocked.
- Area for improvement: the machine learning tasks were added later in the semester and would benefit from earlier integration into the pipeline design phase.

10.3 Future Extensions

- Task 1: extend to individual job description pages to extract explicit required technologies for richer skill analysis
- Task 2: apply VADER or transformers-based sentiment analysis to classify headlines as positive, neutral, or negative
- Task 3: replace price simulation with live daily scraping via a scheduler (APScheduler or cron) for real price tracking
- Task 4: extend to weather forecasting using ARIMA or Facebook Prophet on the 30-day historical series
- Task 5: incorporate vaccination data and healthcare capacity metrics for a more complete epidemiological risk model

References

- JustJoin.it — Polish IT Job Portal. <https://justjoin.it>
- books.toscrape.com — Sandbox E-commerce Scraping Site. <https://books.toscrape.com>
- Open-Meteo Historical Archive API. <https://open-meteo.com/en/docs/historical-weather-api>
- disease.sh — Open Disease Data API. <https://disease.sh>
- Pedregosa et al. (2011). Scikit-learn: Machine Learning in Python. JMLR, 12, 2825–2830.
- McKinney, W. (2010). Data Structures for Statistical Computing in Python. Proceedings of the 9th Python in Science Conference.
- Richardson, L. (2007). Beautiful Soup Documentation. <https://www.crummy.com/software/BeautifulSoup/>
- WSB University — Collaboration Policy for Team Data Science Projects (Course Documentation, 2025/2026)

